CMSC132: Interfaces

Chapter 1

By Naren Ayinala

May 2026

Contents

1 What Is an Interface? 2

1.1 Defining an Interface and Implementing It 2

1.2 Interface Variables and Default Methods 2

2 Objects, References, and Variables 3

2.1 Object References and Aliasing 3

2.2 Primitive, Reference, Local, and Instance Variables 3

2.3 The this Keyword 4

3 The Comparable Interface 4

3.1 Implementing compareTo() 4

3.2 Practice and Check Your Understanding 4

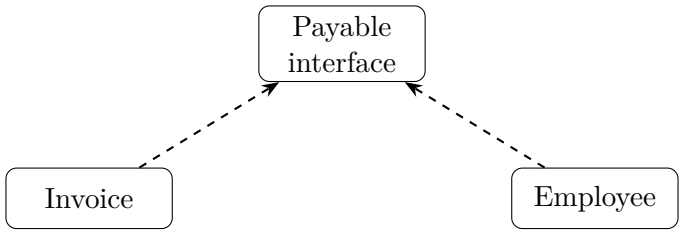
1 What Is an Interface?

An interface is a **contract**: it specifies which operations an implementing class promises to provide. Think of a charging port: the user needs to know the supported connection, not the electronics inside the charger. In Java, the interface is a reference type, not a concrete object you can instantiate directly. Interfaces are especially useful when several unrelated classes share a capability.

1.1 Defining an Interface and Implementing It

Use `interface` to declare the contract and `implements` in the concrete class. Interface methods without a body are implicitly `public abstract`; implementing methods must be public. Interfaces can also have `default`, `static`, and private helper methods (modern Java).

```
interface Payable { double amountDue(); }
class Invoice implements Payable {
    private final double price;
    Invoice(double price) { this.price = price; }
    @Override public double amountDue() { return price; }
}
class Employee implements Payable {
    @Override public double amountDue() { return 1200.0; }
}
class Demo {
    public static void main(String[] args) {
        Payable p = new Invoice(35.5);
        System.out.println(p.amountDue()); // 35.5
        p = new Employee();
        System.out.println(p.amountDue()); // 1200.0
    }
}
```



1.2 Interface Variables and Default Methods

Every field declared in an interface is implicitly `public static final`, meaning a constant rather than an instance field. A default method supplies reusable behavior, but the implementing class can override it. A class can implement multiple interfaces, unlike extending multiple classes.

```
interface Identifiable {
    int MAX_LENGTH = 30; // public static final
    String id();
    default boolean hasId() { return !id().isEmpty(); }
}
class Student implements Identifiable {
    private String studentId;
    Student(String id) { studentId = id; }
```

```
    public String id() { return studentId; }
}
```

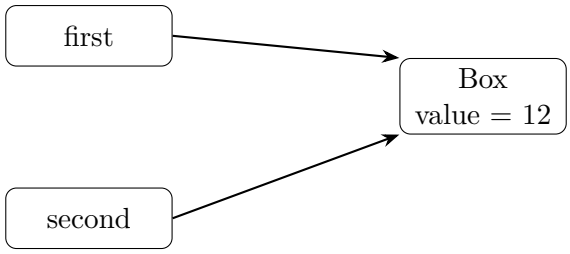
2 Objects, References, and Variables

A class is a blueprint; an object is an instance created from it. Java variables of class or interface type store *references* to objects, not the object data itself. Two references may point to the same object, which explains why modifying through one reference can be observed through the other.

2.1 Object References and Aliasing

```
class Box { int value; }
Box first = new Box();
first.value = 7;
Box second = first; // no new Box was created
second.value = 12;
System.out.println(first.value); // 12
Box third = new Box(); // distinct object
System.out.println(first == second); // true
System.out.println(first == third); // false
```

The operator `==` compares reference identity for objects. For logical value equality, classes typically override `equals`. A `null` reference points to no object; dereferencing it causes `NullPointerException`.



2.2 Primitive, Reference, Local, and Instance Variables

Primitive variables (such as `int` and `boolean`) hold primitive values. Reference variables hold references. Local variables live within a method or block and must be assigned before use; instance fields belong to each object and receive default values if not initialized. Static fields belong to the class, shared among its instances.

```
class Counter {
    private int count; // instance field: starts at 0
    private static int objects; // shared static field
    Counter() { objects++; }
    void increment() {
        int change = 1; // initialized local variable
        count += change;
    }
    int getCount() { return count; }
}
```

2.3 The this Keyword

Inside an instance method or constructor, `this` refers to the current object. It is commonly used to distinguish fields from parameters with the same name. `this(...)` invokes another constructor in the same class and must be its first statement. `this` cannot be used in a static context.

```
class Point {
    private int x, y;
    Point() { this(0, 0); }
    Point(int x, int y) { this.x = x; this.y = y; }
    void move(int dx) { this.x += dx; }
}
```

3 The Comparable Interface

`Comparable<T>` defines the *natural ordering* of objects. It contains `int compareTo(T other)`. The sign matters: negative means this comes before other; zero means equal in ordering; positive means after. The exact magnitude is not important. A comparator is different: `Comparator<T>` defines an external or alternative ordering.

3.1 Implementing compareTo()

```
import java.util.*;
class Book implements Comparable<Book> {
    final String title;
    final int pages;
    Book(String title, int pages) {
        this.title = title; this.pages = pages;
    }
    @Override public int compareTo(Book other) {
        int result = Integer.compare(this.pages, other.pages);
        return result != 0 ? result : this.title.compareTo(other.title);
    }
    @Override public String toString() { return title + ":" + pages; }
}
List<Book> books = new ArrayList<>();
books.add(new Book("B", 120));
books.add(new Book("A", 80));
Collections.sort(books); // A:80, B:120
```

Do not write `return this.pages - other.pages` for arbitrary integers: subtraction may overflow. Prefer `Integer.compare`. For sorted sets/maps, an ordering inconsistent with `equals` can create surprising behavior.

3.2 Practice and Check Your Understanding

1. Can you write `new Payable()`? No: it is an interface. **2.** If `Payable p = new Invoice(10)`, can you access `p.price`? No: the field is private and not part of the interface. **3.** Does `compareTo` have to return -1, 0, or 1? No: only its sign is specified. **4.** What is the difference between `implements` and `extends`? The first promises interface behavior; the second inherits a class or extends another interface.